Lab 7 – Files

Topics Covered:

- Files, testing input and output

<u>Part 1: Pre-Lab Questions</u>

Question 1: Which of the following will correctly open the file text.txt for reading?

a) text = open("text.txt", "r")
b) text = open("text.txt")
c) text = open("text.txt", "read")
d) Both a) and b)

Question 2: Which of the following will correctly open the file mytext.txt for writing?

a) text = write("mytext.txt", "w")
b) text = write("mytext.txt")
c) text = open("mytext.txt", "w")
d) text = open("mytext.txt")

Question 3: Take a look at the following code segment and text file. What will be the output of the following code segment?

Code Segment	text.txt

`text = open("text.txt", "r")`
`for line in text:`
`  print(line)`
`text.close()`
	`cat`
	`dog`
	`monkey`

a) catdogmonkey

b) cat
dog
monkey

c) cat

dog

monkey

Question 4: Take a look at the following code segment and text file. What will be the output of the following code segment?

Code Segment	text.txt

`text = open("text.txt", "r")`
`for line in text:`
`  print(line, end="")`
`text.close()`
	`cat`
	`dog`
	`monkey`

a) catdogmonkey

b) cat
dog
monkey

c) cat

dog

monkey

Question 5: Take a look at the following code segment and text file. What will be the output of the following code segment?

Code Segment	text.txt

`text = open("text.txt", "r")`
`for line in text:`
`  print(line.rstrip())`
`text.close()`
	`cat`
	`dog`
	`monkey`

a) catdogmonkey

b) cat
dog
monkey

c) cat

dog

monkey

Question 6: Suppose a text file contains the lines:

Train

Car

Using the readline() function would return what string for the first call:

a) Train
b) Train\n
c) TrainCar
d) Train Car

Question 7: Suppose the text file data.txt contains the lines:

3,1,4

1,3,7

8,5,2

Determine the output if the following code is executed to work with the above data.txt file.

```
file = open("data.txt", "r")
for line in file:
    v = 0
    line = line.strip()
    nums = line.split(",")
    for n in nums:
        v += int(n)
    print(v, end=' ')
file.close()
```

a) 12 9 13
b) 8 11 15
c) 8, 5, 2
d) 3 1 4 1 3 7 8 5 2
e) It would produce an error

Lab 7 – PART 2

1. Identify errors and fix the code.

Look at the following code segment. This reads from a file called text.txt which has only one line. The program should read this line and split it into individual words; these words should be written to the screen and to the file myfile.txt with each word on its own line. There are 4 errors in this code; find them and correct the code.

```
text = open("text.txt","r")
myfile = open("myfile.txt","r")
line = text.read
words = line.split()
for word in words:
    print(word + "\n")
    myfile.write(word)

# Close the files
text.close()
myfile.close()
```

2. Write the following programs

Read over the hints on the last page before you start coding!

Write a Python program that reads a tab-separated file containing sales data, then calculates the total sales per product and the highest sales day for each product.

Input:

The program should read from a file called sales_data.tsv. Each line in the file contains a product name, the day of the week, and the number of units sold on that day, **separated by tabs**. For example:

```
Apple     Monday     120
Banana     Tuesday     80
Apple     Wednesday     150
Orange     Monday     90
Banana     Friday     60
Apple     Friday     200
```

```
Orange	Thursday	130
```

Note that the spaces between the words/numbers above are a single tab character. You can assume that the file will contain between 1 and 1000 lines.

The product names are not limited to the values shown above, they can be any string that does not contain a tab (\t) or a line break (\n).

Output:

Your program should print the following:

1. The total number of units sold for each product.
2. The highest sales day for each product, including the number of units sold on that day.

Constraints:

- Assume that the file is formatted correctly (i.e., each line contains a product name, day of the week, and number of units sold). You don't need any error checking for this.
- Handle cases where there is only one entry for a product.
- If two or more days have the same highest or lowest sales for a product, choose the day that appears first in the file.

Example:

Given the contents of sales_data.tsv (you can find a copy of this on OWL):

```
Apple	Monday	120
Banana	Tuesday	80
Apple	Wednesday	150
Orange	Monday	90
Banana	Friday	60
Apple	Friday	200
Orange	Thursday	130
```

Your program should output:

```
Total sales per product:
Apple: 470 units
Banana: 140 units
Orange: 220 units

Highest sales days:
Apple: Friday (200 units)
Banana: Tuesday (80 units)
Orange: Thursday (130 units)
```

Hints:

- This is a complex problem, try breaking it down into smaller parts and solve one part at a time debugging and testing as you go. Start with just reading the file in and splitting it into the three values on each line.
- When you are testing your program, make sure the sales_data.tsv is in the same directory/folder as your python program.
- Be careful if you edit sales_data.tsv in PyCharm, there must be tabs between the values, but PyCharm might input spaces rather than tabs if you hit the tab key. The sales_data.tsv file on OWL should already have tabs inserted correctly.
- It might be helpful to use a dictionary where the keys are product names, and the values are a list containing the total sales, the name of the day of the week with the highest sales, and the number of sales on the highest day.
    - **Example:**

      ```
      product_sales = {}
      …
      product_sales[product] = [units_sold, week_day, highest_sales]
      ```

    - Where **product** is the name of the product, **units_sold** are the total number of units sold for this product, **week_day** is the day of the week is the largest number of sales for this product, and **highest_sales** is the number of sales on the day with the highest sales for that product.

- Remember to use the .strip() method to remove any extra line breaks or whitespace at the end of each line.
- You can use the .split() method, such as .split('\t'), to help separate the product name, day, and units sold.
- To start, you can initialize the highest sales to a low value (like -1) for each product and update it when you encounter higher sales for the product. Alternatively, you could start the highest sale for a product as the first value you find for that product and update it when you encounter a higher sales day for that product.
- Add the units sold to the total sales for each product as you read through the file.
- Make sure to handle the case where a product appears multiple times by updating its total sales and highest sales day each time you encounter it. Also account for the case where you encounter a product for the first time.
- After processing the file, iterate through the dictionary to print the results for each product.